

Article

A Malware-Detection Method Using Deep Learning to Fully Extract API Sequence Features

Shuhui Zhang ^{1,2,*} , Mingyu Gao ^{1,2}, Lianhai Wang ^{1,2} , Shujiang Xu ^{1,2}, Wei Shao ^{1,2} and Ruixue Kuang ^{1,2}

¹ Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250353, China; wanglh@sdas.org (L.W.); xushj@sdas.org (S.X.); shaow@sdas.org (W.S.); kuangrx@sdas.org (R.K.)

² Shandong Provincial Key Laboratory of Computer Networks, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China

* Correspondence: zhangshh@sdas.org

Abstract: Due to the rapid emergence of malware and its greater harm, the successful execution of malware often brings incalculable losses. Consequently, the detection of malware has become increasingly crucial. The sequence of API calls in software embodies substantial behavioral information, offering significant advantages in the identification of malicious activities. Meanwhile, the capability of automatic feature extraction by deep learning can better mine the features of API call sequences. In the current research, API features remain underutilized, resulting in suboptimal accuracy in API detection. In this paper, we propose a deep-learning-based method for detecting malware using API call sequences. This method transforms the API call sequence into a grayscale image and performs classification in conjunction with sequence features. By leveraging a range of deep-learning algorithms, we extract diverse behavioral information from software, encompassing semantic details, time-series information, API call frequency data, and more. Additionally, we introduce a specialized neural network framework and assess the impact of pixel size on classification effectiveness during the grayscale image-mapping process. The experimental results show that the accuracy of our classification method is as high as 99%. Compared with other malware-detection techniques, especially those based on API call sequences, our method maps API call sequences to gray image analysis and has higher detection accuracy.

Keywords: deep learning; machine learning; malware detection; visualization; malware classification



Academic Editors: Manuel Mazzara and Andreas Mauthe

Received: 13 November 2024

Revised: 25 December 2024

Accepted: 30 December 2024

Published: 3 January 2025

Citation: Zhang, S.; Gao, M.; Wang, L.; Xu, S.; Shao, W.; Kuang, R. A Malware-Detection Method Using Deep Learning to Fully Extract API Sequence Features. *Electronics* **2025**, *14*, 167. <https://doi.org/10.3390/electronics14010167>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the rise of malware, effective detection technology is essential. According to a recent report [1], by 2023, the total number of malware and potentially unwanted applications (PUA) exceeds 1.2 million, malware comprising more than 80% of this aggregate. The past few years have witnessed a surge in the frequency of malware attacks [2], with over 300,000 new malware instances emerging daily and approximately 3.5 malware instances per second. Malware often causes serious threats such as privacy invasion, data leakage, system crashes, and data corruption, and is extremely harmful. Consequently, it is necessary to design a more automated and highly detected malware-detection method.

Currently, several studies have applied machine-learning and deep-learning algorithms to malicious code detection. Huang et al. propose a hybrid malware visualization

method that combines static analysis with dynamic analysis [3]. They used cuckoo sandboxes for dynamic analysis, converting the analysis results into visualizations. The images obtained were used to train the neural network, and two different models were constructed, with an accuracy of 82.5% and 92.5%, respectively. Deep-learning algorithms offer significant advantages in image classification, enabling automated and efficient learning of texture features for high-precision classification. However, the method of directly extracting features ignores the behavior information of malware, and it is difficult to extract key features. Shenderovitz et al. propose Bon-APT, which segments API(Application Programming Interface) calls during dynamic analysis to study Advanced Persistent Threat (APT) behavior. They tested on a large dataset and showed good results in detecting and attributing APTs [4].

In current existing works, whether using visualization or API call sequences as features, these approaches are often insufficiently explored. Li et al. [5] proposed a method for extracting semantic information from API call sequences. They extracted behavioral details, operation objects, operation goals, and other information from API names. Using deep-learning techniques, they integrated these features for classification, achieving an accuracy of 97.31%. However, they focused solely on mining the features of the sequences themselves. In API call sequences, the frequency of occurrence of a specific API and the statistical information of its call relationships are also crucial features that have a significant impact on detection results. These statistical features were not considered in their approach. On the other hand, research based on visualization often directly converts binary files or code snippets into images. For example, Vasan et al. [6] proposed the IMCFN model, which converts malware binary files into RGB images and feeds them into a training model, achieving an accuracy of 98.82%. However, they use a direct sequence modeling approach, focusing solely on image visualization. This may result in the loss of crucial structural and semantic information that is essential for malware detection. Furthermore, converting code into images may not fully capture the complex relationships between instructions or API calls, therefore limiting the model's ability to generalize to various types of malware.

In this paper, considering the advantages and limitations of deep learning in image classification methods, we propose a malicious code detection method based on deep learning and API call sequences. This method employs deep-learning algorithms to comprehensively extract features from API call sequences. We convert API sequences into images, extracting sequence feature information from a macro perspective while using the detailed features of the text sequence as a supplement. This approach overcomes the limitations of single-feature extraction methods and highlights the complementary advantages of image and semantic features. It not only expands the feature space but also enhances the expressive power of the classifier. During the grayscale image-generation process, we optimized the mapping strategy, resulting in a more uniform pixel-value distribution in the final image, ensuring that the CNN can more effectively capture the features and achieve high accuracy in malware detection. The proposed method leverages API call sequences for intrusion detection, offering a detailed insight into software behavior. By analyzing API interactions, the approach identifies malicious patterns that are indicative of potential threats. This enables a more granular detection compared to traditional methods, effectively distinguishing between benign and malicious behaviors. Moreover, in terms of monitoring and privacy, our approach only collects each API of a process and does not collect the parameters of the API, nor does it collect other user data, thus ensuring that user privacy is not involved. Specifically, all API call sequences are obtained by running samples in a virtual sandbox. The environment is designed to closely simulate real user conditions, but it does not contain any real user information. In the sandbox report, we only extract API call sequences, and throughout the analysis, no user information that might appear in

API parameters is involved—only the APIs themselves are considered. Once unexpected user information-related content appears we will also pseudonymize it.

In summary, the contributions of our work are as follows:

- (1) Concerning the present challenge of inadequate utilization of API call sequences, we build a neural network model based on deep learning to extract API call sequence features through TextCNN, CNN, Bi-LSTM, and other algorithms. This model aims to extract API call sequence features from both semantic and graphical perspectives to enhance malware detection.
- (2) We propose a malware-detection method based on deep learning and API call sequences. The API call sequences are mapped into grayscale images and classified by combining semantic features, making full use of the advantages of deep learning in image feature extraction and effectively extracting malware behavior information.
- (3) We present our experimental results for binary and multiclass classification on a large dataset of malicious and benign samples. Our proposed method achieves an accuracy of 99%, which is better than the results of previous studies that feature API call sequences for malware detection.

2. Related Work

Traditional antivirus software quickly detects malware through static analysis without executing samples. It commonly depends on signature matching algorithms for malware classification [7]. However, this method is limited to matching existing signatures in the database, making it prone to failure when encountering unknown malware, variants, and challenges posed by code obfuscation techniques. Recognizing the limitations of static analysis, security experts prefer to incorporate dynamic analysis into the malware-classification process. Next, we will introduce the malware-detection methods discussed in this paper.

2.1. Behavior-Based Malware-Detection Approaches

Behavior analysis stands out as a widely used dynamic analysis method for detecting malicious code. It monitors and analyzes program behavior at runtime to identify potential malicious activities. It analyzes both static program characteristics, like file signatures, and dynamic behaviors, including API call sequences, file operations, network communication, and memory access. The method efficiently detects both unknown malicious code and its variations, providing a potent and flexible way for malware detection.

Qiang et al. employed control-flow tracing to extract behavioral features [8]. They converted control-flow traces into byte sequences and utilized a combination of Convolutional Neural Networks and Long Short-Term Memory to construct a classifier. Zhang et al. employed deep learning and memory forensics technology for malicious code detection [9]. They selected binary fragments with different lengths and positions and input them into the neural network, obtained an accuracy of 97.48%, and could detect fileless attacks. Zhou et al. introduced an introspective framework utilizing the Intel management engine to gather memory information, transmitting it to a remote machine [10]. This framework identifies malicious attacks by verifying memory integrity on the target machine and monitoring the runtime state of the host system. Yang et al. collected audit logs generated by APT attacks, built a traceability graph, divided it into snapshots according to the timestamp, and then performed graph embedding representation for each snapshot, used the embedded information of snapshots of benign activities to train a prediction model, and detected abnormal snapshots [11]. Wang et al. used the traceability graph to represent the entire behavior of malware, and extracted the “causal path” from the graph, and judged whether it was maliciously attacked by detecting the abnormal degree of the path [12].

Tan et al. employed MTB in the microcontroller to monitor the execution process and construct a control-flow graph for analyzing software behavior in the trusted execution environment [13]. This approach ensures control-flow integrity, enabling the detection of malware attacks in embedded systems.

Detection methods grounded in behavior analysis play a pivotal role in safeguarding computer systems and data by identifying indicators of abnormal behavior, particularly in areas like network security, antivirus solutions, and threat detection. However, behavior analysis often involves monitoring and recording system behavior data, raising concerns about user privacy. Additionally, the complexity introduced by the diversity and variants of malicious code further challenges behavior analysis.

2.2. Visualization-Based Malware-Classification Approaches

Visualization-based malware-detection methods combine data visualization techniques with malware analysis to reveal potential malicious behaviors, features, and patterns more intuitively. Deep-learning algorithms have shown strong superiority in image classification and feature extraction. By employing these algorithms to extract visual features, the detection system gains access to more precise and easily classifiable features, therefore improving overall recognition accuracy.

Nataraj et al. initially applied visualization technology to malicious code detection [14]. They employed static analysis to extract the text block in malicious code, visualizing it as a grayscale image based on binary sequence rules. The GIST algorithm was then utilized to extract image features, followed by classification using the K-nearest neighbor algorithm. This method only uses static analysis to obtain the static content of malware and does not consider dynamic behavior information, which cannot cope with code obfuscation techniques. Bensaoud and Kalita [15] proposed a novel model for malware classification utilizing API calls and opcodes. They integrate Convolutional Neural Networks with Long Short-Term Memory architecture. By transforming features into N-gram sequences and exploring various deep-learning architectures, high detection results were achieved. Vasan et al. proposed the IMCFN model, which converted the malware binary into RGB images and sent them to the training model to obtain the classification results [6]. This study compared the performance of the proposed method with different pre-trained Convolutional Neural Network models VGG-16, ResNet-50, and Google InceptionV3 and obtained an accuracy of 98.82%. Shah et al. separated color channels into individual datasets, reduced dimensions, and removed noise using Discrete Wavelet Transform. They used the red channel dataset, and extra tree classifiers achieved an accuracy of 98.37% [16]. Shaukat Kamran et al. visualized Portable Executable (PE) files as color images, used deep-learning models to extract features from color images, and finally used Support Vector Machine (SVM) to detect the extracted features. This method integrates deep learning and machine learning and possesses high scalability and efficiency [17]. Islam et al. put forward a method for the visualization of Java malware visualization. They transformed Java bytecode files into interprocedural control-flow graphs using API [18]. Subsequently, these graphs were visualized as grayscale images. Combining these images with a set of behavioral features, they utilized Convolutional Neural Networks (CNN) for classification, achieving a final accuracy of 98.4%. This represented a substantial improvement compared to classification methods without visualization.

Although the visual malware-detection technology combined with deep learning has great advantages in feature extraction and dealing with new malware, it also faces the problem of information loss. In the process of representing malicious as an image, some information may be lost or ambiguous. At the same time, most grayscale images

in existing studies are obtained by static analysis (such as bytecode files), which cannot contain dynamic behavior information.

2.3. API Call Sequence-Based Malware-Classification Approaches

In this work, we are interested in using API call sequences for malware detection because API call sequences contain rich behavioral information and can effectively reflect the behavioral purpose of the software. Traditional methods for extracting features from API call sequences rely on factors like the frequency, order, and call depth of API functions. These methods often involve manual extraction and analysis by experts, overlooking the temporal relationship between API call sequences. The application of deep learning solves the disadvantages of traditional feature extraction methods in API call sequences, and the extraction process is more automated while extracting deeper features.

At present, malware detection based on API call sequences has been studied by many people. Jha et al. use a Recurrent Neural Network (RNN) for malware detection. They used three schemes to represent API features: hot encoding feature vector, random feature vector, and Word2Vec feature vector. A comparative evaluation of three different solutions was conducted. The results indicate that Word2Vec achieves the highest performance among the solutions tested [19]. Gómez et al. derived and released three datasets [20]. They conducted a comprehensive examination of the most relevant static characteristics associated with malware samples using various machine-learning and deep-learning algorithms. By utilizing features such as API calls, permissions, and intents, they achieved high detection performance in Android malware detection. Amer and Zelinka used the word embedding method to represent each API name as a vector containing context information to achieve malware detection [21]. The disadvantage of this method is that it only considers the API name and does not conduct in-depth research on the API. Nige et al. [22] proposed a model for detecting malicious behaviors in power system edge-side applications. It combined rule matching and deep learning, using API call sequences and mining frequent sequences with PrefixSpan. TextCNN was employed for detection, showing effectiveness in discerning malicious behaviors. Saqib et al. introduced the SPM algorithm, abstracting API sequences into integer sequences [23]. They performed sequential pattern mining to explore features like frequent API call sequences and sequence relationships. Machine-learning technology was then applied to classify the discovered features. Li and Lin proposed CAFTrans, a Transformer-based model for malware detection. They used CNNs and LSTMs to capture API relationships. CAFTrans achieved good performance on the mal-api-2019 dataset with an F1 score of 0.65252 and an AUC of 0.8913 [24]. However, this approach utilized only a limited number of features from the extensive set of extracted features, neglecting the full utilization of API call sequence features.

In our study, we analyze API call sequences through deep-learning algorithms, visualize API call sequences as grayscale images, and extract semantic features of API call sequences through a variety of neural networks. This method not only plays the advantages of deep learning in image classification, but also includes the dynamic behavior characteristics of software. We will describe our method in the next section.

3. Neural Network Algorithm

This section describes the neural network algorithm used in our study.

The Convolutional Neural Network (CNN) primarily finds application in image recognition. It automatically learns and extracts image features through components such as convolutional layers, pooling layers, and fully connected layers. The convolution operation can effectively capture local information by setting the convolution kernel to slide in the image. The pooling layer, encompassing Max pooling and average pooling, serves to diminish

data dimensionality. Finally, classification is achieved through the fully connected layer. This architectural design empowers CNNs to excel in handling two-dimensional data, particularly images. The convolution operation is mathematically represented as follows:

$$\text{out}(N_i, C_{\text{out}_j}) = B(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \text{input}(N_i, k) \quad (1)$$

where N is the batch size, C is the number of channels, and B is the bias value of the neural network. The specific settings of convolutional kernels vary across different levels in the CNN cascade. Typically, shallower convolutional layers use relatively small kernel sizes, while deeper convolutional layers prefer wider kernels for more complex feature extraction. Figure 1 demonstrates the convolution process: Suppose that we have a single-channel 3×3 input feature map and a 2×2 convolution kernel. First, the convolution kernel performs element-wise multiplication with a local 2×2 region of the input feature map and then sums the results to obtain the convolution result for the first part. This process is repeated until the convolution kernel slides over the entire input feature map.

After the convolution operation, the length and width of the output feature map may be decimal, and the output size is calculated as follows:

$$\text{output size} = \left\lfloor \frac{\text{input size} - \text{kernel size} + 2\text{padding}}{\text{stride}} \right\rfloor + 1 \quad (2)$$

where input size represents the length or width of the input feature map, kernel size represents the size of the convolution kernel, padding represents the size of the padding, and stride represents the step size of the convolution. Finally, the output size is obtained by rounding the result down and adding 1. Similarly, for the 3×3 feature map shown in Figure 1, with padding set to 0 and a 2×2 convolution kernel with a stride of 1, the output size is calculated as follows: $\text{output size} = \frac{3-2+2 \times 0}{1} + 1 = 2$. Therefore, the size of the output feature map will be 2×2 .

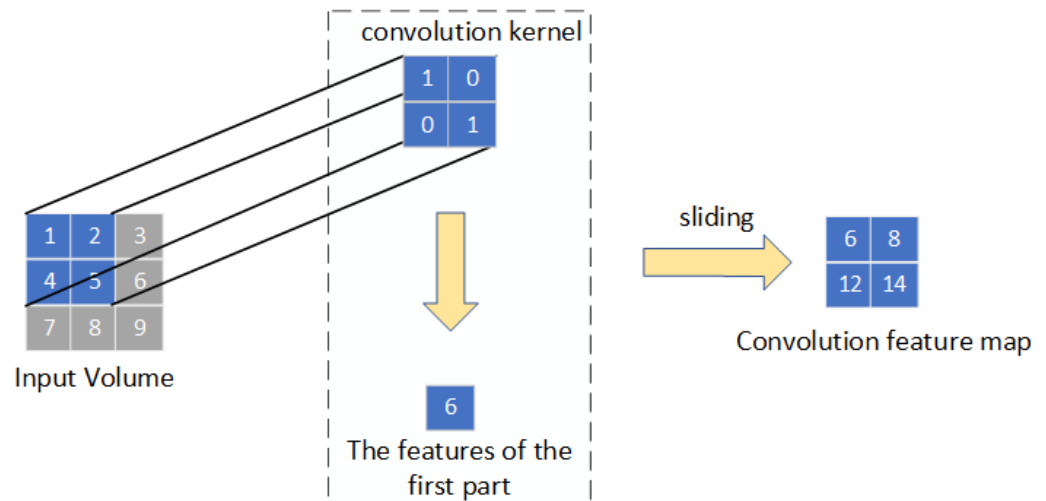


Figure 1. Convolution process.

TextCNN is a variant of Convolutional Neural Network (CNN) in text classification tasks, with a more concise and clear overall structure [25]. Similar to image processing, TextCNN employs a convolution operation to capture local features in the text and subsequently reduces the dimension of these features through pooling operations. This enables the model to understand the key information at different locations in the text and thus better perform the classification task. In TextCNN, the text sequence is represented as an embedding vector. Convolutional operations are applied at different positions using

convolution kernels, followed by pooling operations to extract the most significant features. These features go through a fully connected layer and are finally classified.

Long Short-Term Memory networks (LSTM) are a variant of recurrent neural networks (RNN) specifically designed to process sequential data. LSTM effectively solves the long-term dependence problem in traditional RNNs through the gate structure, including the input gate, forget gate, and output gate. The input to LSTM is a sequence $(x_1, x_2, x_3 \dots x_n)$, which is computed as follows:

The first step is to decide what information we will discard from the cell state. This decision is made using a structure called the “forget gate”, which is calculated as follows:

$$f_t = \sigma(W_{fh}h_{t-1} + W_{fx}x_t + b_f) \quad (3)$$

where W represents the weight matrix, h_{t-1} represents the output of the previous time step, x_t represents the current output, b is the offset vector, σ is the nonlinear transformation. The forget gate will read the previous output and the current input. After the Sigmoid nonlinear mapping, the output of the forget gate at the current time step is obtained.

The second step goes through the input gate, which determines what kind of new information is stored in the cell state, which is calculated as follows:

$$i_t = \sigma(W_{ih}h_{t-1} + W_{ix}x_t + b_i) \quad (4)$$

$$\tilde{C}_t = \tanh(W_{ch}h_{t-1} + W_{cx}x_t + b_c) \quad (5)$$

The parameters have a similar meaning to the forget gate, where $\tilde{C}_t$ is an intermediate state.

The third step updates the old cell state, i.e., C_{t-1} is updated to C_t , which is calculated as follows:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (6)$$

The fourth step goes through the output gate to determine what value o_t to output and obtain the hidden layer output h_t , which is formulated as follows.

$$o_t = \sigma(W_{oh}h_{t-1} + W_{ox}x_t + b_o) \quad (7)$$

$$h_t = o_t * \tanh C_t \quad (8)$$

Finally, we select the necessary components for output, ensuring that the output of the hidden layer retains past information.

4. Overview

4.1. Framework

Figure 2 represents our complete framework diagram of the classification process. The system takes PE files as input and eventually uses supervised deep learning to implement both binary and multi-classification tasks for malware. Our approach comprises three main modules: the API sequence preprocessing module, the graph feature extraction module, and the semantic feature extraction module. Below, we provide a brief explanation of these three modules.

The API sequence preprocessing module: This module primarily extracts the API call sequence in the sample program and assigns a unique number to each API for easy computer identification and processing. Finally, the redundant API is deleted to obtain the API sequence to be analyzed.

The graph feature extraction module: This module visualizes the preprocessed API call sequence as a grayscale graph. Leveraging the strengths of Convolutional Neural Networks in graph recognition, it extracted features such as the number of pairwise calls, relationships between APIs, and the frequency of API usage. These features provide insights into malicious behavior to a certain extent. Finally, the key features were extracted through the attention mechanism.

The semantic feature extraction module: In this module, algorithms suitable for text sequence analysis, such as TextCNN and LSTM, are applied to the preprocessed API call sequence to extract the semantic information of the API call sequence.

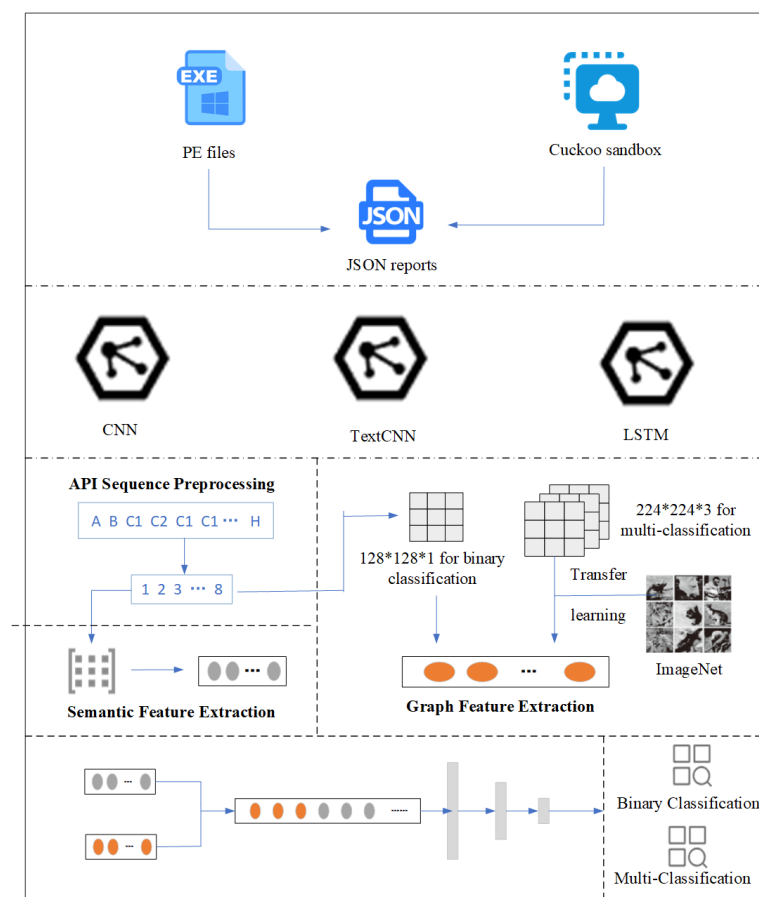


Figure 2. Framework Overview.

Finally, the two features are classified on the Convolutional Neural Network classifier to classify the input PE file. We will introduce our method in detail in Section 4.

4.2. Transfer Learning

Transfer learning has become a hot topic in deep learning and artificial intelligence. It is a method in machine learning that allows a model to apply knowledge learned from one task to another related task. This approach is especially useful when data are scarce, as it reduces the need for large amounts of labeled data. Transfer learning is defined as follows.

Domain D consists of feature space X and marginal probability $P(x)$, where $x = \{x_1, x_2 \dots x_n\} \in X$. Transfer learning is applied to a task that has only one source domain D_s and one target domain D_t . Given a domain $D = \{X, P_x\}$, a task is represented as $T = \{Y, f(\cdot)\}$, where Y represents the label space and $f(\cdot)$ represents the objective function. The objective function cannot be observed directly but can be learned from training samples.

Transfer learning is usually applied in tasks where there is only one source domain and one target domain. Source domain $D_s = \{(x_{s1}, y_{s1}), (x_{s2}, y_{s2}) \dots (x_{sn}, y_{sn})\}$, where

$x_{si} \in X_s$ indicates the sample of the source domain. $y_{si} \in Y_s$ indicates the label of the source domain. Target domain $D_t = \{(x_{t1}, y_{t1}), (x_{t2}, y_{t2}) \dots (x_{tn}, y_{tn})\}$, where $x_{ti} \in X_t$ indicates the sample of the target domain, $y_{ti} \in Y_t$, indicates the label of the target domain. In general, the relationship between the sample number n_s of the source domain and the sample number n_t of the target domain is as follows: $1 \leq n_t \ll n_s$.

In our proposed model, we apply transfer learning to multiclass classification tasks. Specifically, D_s represents the ImageNet dataset, and D_t represents the dataset we collected. We use the VGG16 network and PyTorch to implement transfer learning in this experiment.

5. Proposed Method

5.1. The API Sequence Preprocessing Module

The first step in our approach is API call sequence preprocessing. To obtain the API call sequence, we employ the CAPE sandbox in the virtual machine to isolate the PE files in the sandbox for batch running. After executing each sample, the virtual machine reverts to the pre-saved normal snapshot state. It then proceeds to run the next sample, repeating this process until all benign and malicious samples have been executed. The results are stored in separate folders to distinguish between malicious and benign samples. Upon completion of sample execution, the Host Machine generates an analysis report for each sample. The report contains rich sample characteristics, including program execution snapshots, API and DLL calls, strings, stored files, process and thread information, and so on. Our focus lies in the API call sequences, and for detection purposes, we extract the API call segment. Parsing the resulting JSON file involves pinpointing nodes associated with dynamic behavior. By identifying API call information at each timestamp and navigating the relevant JSON nodes, we can organize the API calls in chronological order, resulting in the extraction of the API call sequence. Algorithm 1 illustrates the process of retrieving API sequences. We use the sandbox report paths as input and extract the API call sequences from each report file.

Algorithm 1 Get API Sequences

Require: Report Path from Cuckoo

Ensure: API Call Sequences

```

1:  $Calls \leftarrow NULL$ 
2: for each folder in Path do
3:   if 'report.json' exists then
4:      $jsData \leftarrow$  load JSON data from 'report.json'
5:      $repDict \leftarrow NULL$ 
6:      $callList \leftarrow NULL$ 
7:     if behavior not in  $jsData$  then
8:       skip to next iteration
9:     end if
10:     $md5 \leftarrow$  find md5 from  $jsData$ 
11:     $repDict['md5'] \leftarrow md5$ 
12:     $call \leftarrow jsData['behavior']['processes']['calls']$ 
13:    if call exists then
14:      append  $call['api']$  to  $callList$ 
15:    end if
16:     $repDict['callList'] \leftarrow callList$ 
17:    append  $repDict$  to  $Calls$ 
18:  else
19:    skip to next iteration
20:  end if
21: end for
22: return  $Calls$ 
```

Due to the API call sequence obtained by the program running being too long, if the whole sequence is put into the neural network analysis, it will lead to too large input, and the complexity will increase rapidly. Here, we select a part of API call sequences as training and testing samples. The complete API call sequence often contains numerous consecutive and repetitive patterns. To mitigate interference in the analysis of malicious code, we traverse the entire sequence, eliminating continuously repeated subsequences. After deleting the subsequence, as shown in Table 1. The resulting API call sequence possesses a substantial behavioral span and an optimal sequence length. The research conducted by recent research [26] indicates that malicious activities can be manifested within two minutes of program execution. After our preprocessing, a significant number of consecutive repetitive calls and subsequences were removed. The time span of the first 1000 sequences is sufficiently large to represent malicious behavior. Moreover, many of the malware samples did not have up to 1000 API calls after preprocessing. Therefore, we chose approximately 1000 API calls as the sequence length and will test the impact of different sequence lengths in subsequent experiments. Finally, we counted all the APIs that appeared and created a dictionary, each represented by a unique number.

Table 1. API sequence deduplication.

Complete Sequence	Remove Repetitive Subsequences
NtOpenKey	NtOpenKey
GetSystemTimeAsFileTime	GetSystemTimeAsFileTime
NtQuerySystemInformation	NtQuerySystemInformation
NtAllocateVirtualMemory	NtAllocateVirtualMemory
NtAllocateVirtualMemory	NtAllocateVirtualMemory
NtAllocateVirtualMemory	LdrGetDllHandle
LdrGetDllHandle	

5.2. The Graph Feature Extraction Module

In this module, we represent the preprocessed API call sequence as a grayscale image and employ deep-learning algorithms for texture feature extraction. Convolutional neural networks (CNNs) excel in image feature extraction and classification, so we use Convolutional Neural Networks for feature extraction. When applying CNN for feature extraction, we considered the potential issue of excessive computational resource consumption. We attempted to use a larger number of APIs, but this did not improve classification performance and instead increased the consumption of computational resources. Therefore, we chose to use the first 1000 APIs as the initial dataset to generate grayscale images, aiming to minimize resource consumption. Algorithm 2 presents the algorithm for batching sequences of API calls.

To convert the API call sequence into a 2D matrix, we initialize an $N \times N$ matrix ($Arr[N][N]$), where N represents the number of APIs in the dataset (denoted as X). All matrix values start at 0. During preprocessing, each API is mapped to a unique numeric representation in string form, enabling easy association of 0 to N in the matrix's rows or columns with the corresponding API. Therefore, we can define a mapping from the API call sequence to the grayscale image as follows:

Suppose the number corresponding to A_i is i , and the number corresponding to A_j is j , where A_i and A_j are API names, respectively. Traversing the whole API call sequence, if there is $A_i \rightarrow A_j$, we increment the value of the matrix $Arr[i][j]$ by n , where n is the grayscale pixel threshold.

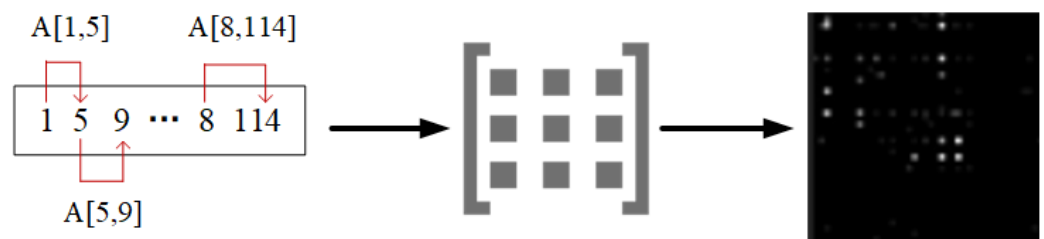
Algorithm 2 Batch Retrieval of API Images**Require:** API Call Sequences, pixel, batchSize**Ensure:** API Images

```

1:  $APIs \leftarrow$  API Call Sequences,  $Pixel \leftarrow$  The increased pixel value per call
2: Initialize  $Arr$  as a  $342 \times 342 (N \times N)$  2D array with zeros
3:  $numBatches \leftarrow \lfloor (len(APIs) + batchSize - 1) \div batchSize \rfloor$ 
4: for each  $batchIndex$  in  $[0, numBatches - 1]$  do
5:    $batchStart \leftarrow batchIndex \times batchSize$ 
6:    $batchEnd \leftarrow \min((batchIndex + 1) \times batchSize, len(APIs))$ 
7:    $imageList \leftarrow NULL$ 
8:   for each  $idx$  in  $[batchStart, batchEnd - 1]$  do
9:      $Seq \leftarrow APIs[idx]$ 
10:    Reset array  $Arr$  to all zeros,  $i \leftarrow 1$ ,  $len \leftarrow len(Seq) - 1$ 
11:    for  $i$  to  $len(Seq)$  do
12:       $api_1 \leftarrow Seq[i - 1]$ 
13:       $api_2 \leftarrow Seq[i]$ 
14:      if  $Arr[api_1 - 1][api_2 - 1] \geq 255$  then
15:         $Arr[api_1 - 1][api_2 - 1] \leftarrow 255$ 
16:      else
17:         $Arr[api_1 - 1][api_2 - 1] \leftarrow Arr[api_1 - 1][api_2 - 1] + Pixel$ 
18:      end if
19:    end for
20:     $GraIMG \leftarrow$  Convert  $Arr$  to grayscale image
21:    Append  $GraIMG$  to  $imageList$ 
22:  end for
23: end for
24: return  $imageList$ 

```

Since the pixel value of a grayscale image is between 0 and 255, once a value in the matrix exceeds 255, we will always treat it as 255. We need to reduce the occurrence of such cases as much as possible, so the value of n should be chosen reasonably to ensure that it is recognizable. In our dataset, each program retains only 1000 API calls post-preprocessing, reducing interference from consecutive calls. This reduction helps minimize interference from consecutive API calls, and the number of calls between two APIs is kept from becoming excessive. If the threshold is too low, the model's discriminative ability will be affected; if it is too high, many call relationships will concentrate around the maximum pixel value of 255, which can hurt classification performance. In our dataset of 1000 API call sequences, we preprocessed the data by removing consecutive duplicates to eliminate meaningless repetitions. We then calculated the frequency of each call pair, with an average of 10 occurrences per pair. To avoid over-concentration near the maximum pixel value of 255, we initially chose a threshold increment between 10 and 25. If we choose to retain longer API sequences, we can consider reducing the value of n appropriately to improve the identification under the premise that it exceeds 255 in a small number of cases. Finally, we construct $N \times N$ grayscale images from the 2D matrix, where each pixel corresponds to the matrix's value, resulting in the API sequence depicted in Figure 3.

**Figure 3.** API sequence map to grayscale image.

Our mapping method constructs the graph by considering both the statistical features of the APIs and the relationships between API calls. These features, particularly the call relationships, are crucial in the feature extraction process from API sequences. Alternative methods, such as direct sequence modeling or graph-based representations, are less effective in capturing these key aspects. The subsequent step involves feature extraction from the grayscale image for classification. In order to better extract the features of the image, we choose to use the CNN for experiments. The input of the model is the grayscale image, and the output is a 1D image feature vector obtained through flattening by the Flatten layer. We employ VGG neural network architecture, utilizing small convolution kernels, Max pooling, the ReLU activation function, and an attention mechanism to extract features. We introduced an attention mechanism within the convolutional layers to enable the model to automatically focus on high-resolution regions of the grayscale image while effectively ignoring potential noise. The attention module assigns higher weights to the important areas of the image that contain more discriminative features while minimizing the impact of irrelevant or noisy regions. This mechanism is applied after the initial convolutional layers, where it dynamically adjusts the feature map activations before passing them to the subsequent layers for further processing. This approach effectively minimizes the negative impact of noise, allowing the model to extract more meaningful and relevant features. The 1D image features obtained through the Flatten layer are used as inputs for the classification module. The grayscale image features in this section are primarily used to reflect statistical characteristics such as the frequency of API call relationships within the API call sequence. In the next section, we will introduce how to extract the textual semantic features.

5.3. The Semantic Feature Extraction Module

In this section, we employ the TextCNN and LSTM algorithms to extract semantic features from API call sequences. TextCNN is a lightweight Convolutional Neural Network that is particularly well-suited for text classification tasks. We chose TextCNN because it effectively captures the semantic features of API call sequences through convolutional kernels of varying sizes, enabling the extraction of multi-scale semantic information. Moreover, TextCNN strikes a favorable balance between computational efficiency and model complexity. In contrast, LSTM is particularly effective at capturing long-range dependencies in sequences. Given the inherent temporal dependencies within API call sequences, LSTM is well-suited for capturing this long-range information while simultaneously reducing the feature dimensionality produced by TextCNN. We also explored alternative algorithms, such as Transformer. Although Transformer models demonstrate superior performance in many natural language processing tasks, they require significantly more computational resources and incur higher time costs compared to TextCNN.

The first step in the semantic feature extraction module is to apply the TextCNN algorithm to the API call sequence to extract multi-scale semantic features. To facilitate 1D convolution, each API transforms a k -dimensional vector. Subsequently, the API call sequence for each sample is transmuted into a two-dimensional matrix, where each row corresponds to an API call. In this process, we considered two methods. Initially, we attempted representation using one-hot encoding for each API, but this method yielded suboptimal extraction results. Subsequently, we consider training the API using the word embedding method to obtain the representation of all word vectors. The vectors obtained through the word embedding method are better suited as inputs for our model. Due to the large number of APIs in the one-hot encoding API vector representation, the dimensions are extensive. Each vector in this method contains too many "0"s, resulting in a sparse two-dimensional matrix with substantial storage overhead. The embedding vector obtained by the word embedding method can not only retain the complex relationship between

APIs but also has a small dimension to store the vector, and the resource consumption of subsequent training is smaller. Therefore, we finally adopt word embedding to obtain the vector representation of each API and convert the API call sequence into a two-dimensional matrix represented by vectors.

TextCNN is a CNN model specifically for text classification. We used $2 \times n$, $3 \times n$, and $4 \times n$ convolution kernels and set the stride to 1. Compared with CNNs suitable for images, TextCNN has greater advantages in feature extraction of one-dimensional data, especially text sequences. Here, we apply TextCNN to a 2D matrix to fully mine the semantic information of API call sequences using convolution kernels of different sizes to obtain fields of view of different widths.

The second step in the semantic feature extraction module is to apply the Long Short-Term Memory network (LSTM) to the three types of features extracted in the first step. LSTM is an improved RNN algorithm, which is specially used to deal with long sequence data. It reinforces long-term dependencies in the sequence, employing three gating mechanisms—input gate, output gate, and forget gate—to selectively incorporate new information and discard previously accumulated information. This approach excels in the training of long sequences. As the API call sequence also exhibits a certain reverse connection, we opt for the Bi-LSTM approach to simultaneously extract features in both forward and backward directions. Three Bi-LSTMs are deployed, each taking one of the three semantic features from the initial step as input. This ensures the extraction of long sequence features from convolved semantic information at different scales. Finally, the outputs from the three Bi-LSTMs are concatenated to obtain the ultimate semantic features. The semantic features are fully integrated with the graph features, leveraging the advantages of both to capture the complexity of the API sequence. These combined features are then fed into the classifier for final categorization.

5.4. Neural Network Architecture and Malware Classifier

In this section, we first introduce the neural network model. Figure 4 is the neural network architecture for binary classification we built. In the image feature extraction module, we fixed the gray image to 128×128 pixels and obtained 9216 flattened feature vectors after multi-layer convolution pooling.

In the semantic feature extraction module, we acquire features under various perspectives by configuring distinct convolution kernels. Subsequently, we employ Bi-LSTM for each feature and concatenate the outputs of the three Bi-LSTMs to form the semantic features.

Next, we introduce the malware classifier, namely the feature classification module. This module integrates the image and semantic features extracted from the preceding two modules, utilizes them as input, and, through a sequence of operations, produces a binary classification outcome—determining whether the binary software is malicious or not. We combine the image features flattened by the Flatten layer with the semantic features extracted by three Bi-LSTMs and input them into multiple fully connected layers. After a series of nonlinear transformations, the sigmoid function is used to output the classification results. To prevent the overfitting problem of the model, we incorporate multiple Drop layers between the fully connected layers. Additionally, we apply five-fold cross-validation to improve the robustness of the model, and the subsequent experimental results are derived from the average of the cross-validation outcomes.

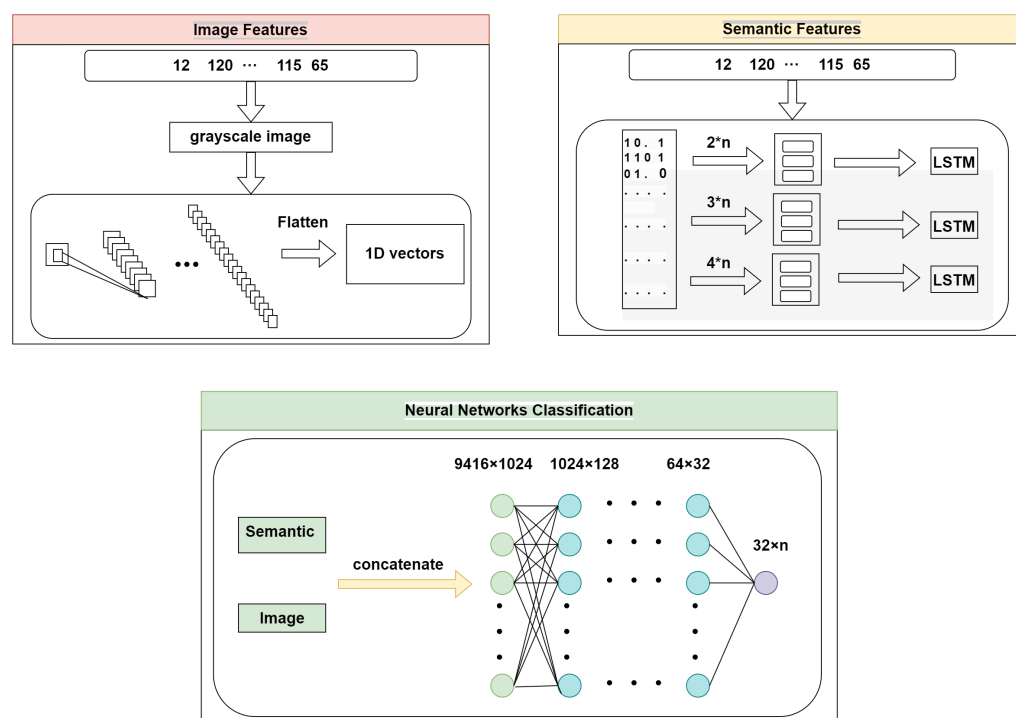


Figure 4. Neural network framework.

6. Experimental Setup

6.1. Experimental Environment and Evaluation

The hardware environment for our training and testing has 16GB memory, a processor of R7-4800H, and a graphics card of NVIDIA GeForce RTX 2060. The Cape sandbox was set up in the Ubuntu 20.04 operating system to segregate the execution of malicious and benign software. We build the Windows operating system in VirtualBox for secure execution of malicious software. Our deep-learning code was executed on the Windows 10 operating system with Python 3.8, Anaconda 4.10, and torch 2.0.1+cu117.

We used two datasets, Dataset A and Dataset B, which were used for binary classification tasks and multi-classification tasks, respectively. For Dataset A, the API call sequence dataset was constructed as follows: Malicious samples were downloaded from malware-sharing websites such as VirusShare and Malshare, and benign download samples were searched from Microsoft's official website. All samples were executed in isolation in the cape sandbox, and the execution time of each sample was set to eight minutes. We analyzed the sandbox report, parsed the JSON file to extract the API call sequence, and obtained the dataset before preprocessing. The final Dataset A consists of 2000 benign software samples and 2000 malware samples. After running them in a sandbox environment, we obtained execution instances for both benign and malicious software. We removed consecutive duplicate sequences and performed preprocessing. The resulting data were then used for a binary classification task to validate whether our method can effectively detect malware. For Dataset B, We used the data collected by Ferhat et al. [27], which was collected by the researchers and validated in VirusTotal as a baseline dataset for various malware API calls in Windows operating systems. Dataset B contains eight common types of malware, including viruses, worms, and Trojans. The specific categories and quantities are shown in Table 2.

The two datasets are independent of each other. Dataset B includes various types of malware, such as worms, Trojans, and spyware, covering a wide range of commonly encountered malicious software in real-world scenarios. These diverse types of malware exhibit distinct behaviors, which helps in capturing different patterns of malicious ac-

tivity. Additionally, we partitioned the dataset, using 60% of the data for training and the remaining 40% for testing. The malware samples in the test set are unseen during training, allowing us to evaluate the model's performance in detecting unknown malware. This approach demonstrates the model's ability to generalize to new, previously unseen malicious samples. We evaluate our work by four evaluation metrics commonly used in malware classification tasks: Accuracy, Precision, F-measure and recall.

Table 2. Malware Family and Samples.

Malware Family	Samples
Spyware	832
Downloader	1001
Trojan	1001
Worms	1001
Adware	379
Dropper	891
Virus	1001
Backdoor	1001

6.2. Experimental Comparison

We first evaluated the overall time consumption of our model on Dataset A. Table 3 shows the time taken from preprocessing to model training completion. Dataset A consists of 4000 samples, including 2000 benign software samples and 2000 malware samples. Our method took a total of 4661 s to complete the training process on Dataset A. During the preprocessing phase, the process of traversing a large number of JSON file nodes, extracting the API call sequences, and performing multiple rounds of duplicate sequence removal resulted in a relatively long processing time of 1260 s. After preprocessing, we effectively reduced the length of the API call sequences while ensuring that they reflected the malicious software behavior, with the image conversion taking only 35 s. During the training phase, the average time per training epoch was 83 s. Overall, this time consumption is quite reasonable and allows the model to be quickly deployed and trained on the target machines.

Table 3. Time Consumption Details.

Module	Times (s)
API Sequence Preprocess	1260
Grayscale Graph Generation	35
Training the model	3366
Total	4661

To explore the influence of each module on the experimental results, we trained and tested four models using only image features, only semantic features, no attention mechanism and our proposed method. For image features, we mapped the API call sequence to a grayscale image as the input of the Convolutional Neural Network to extract features. The output is then fed into the Flatten layer, transforming it into a one-dimensional vector. Finally, the classification results are obtained through several fully connected layers and sigmoid functions. For semantic features, the API call sequence serves as the neural network input. Features between sequences are extracted using three convolution kernels of varying scales, and each set of features is processed by individual Bi-LSTMs. Finally, after concatenation of the output of Bi-LSTM, the classification results are obtained through the fully connected layer and the sigmoid function. Figure 5 shows the metrics for the four models.

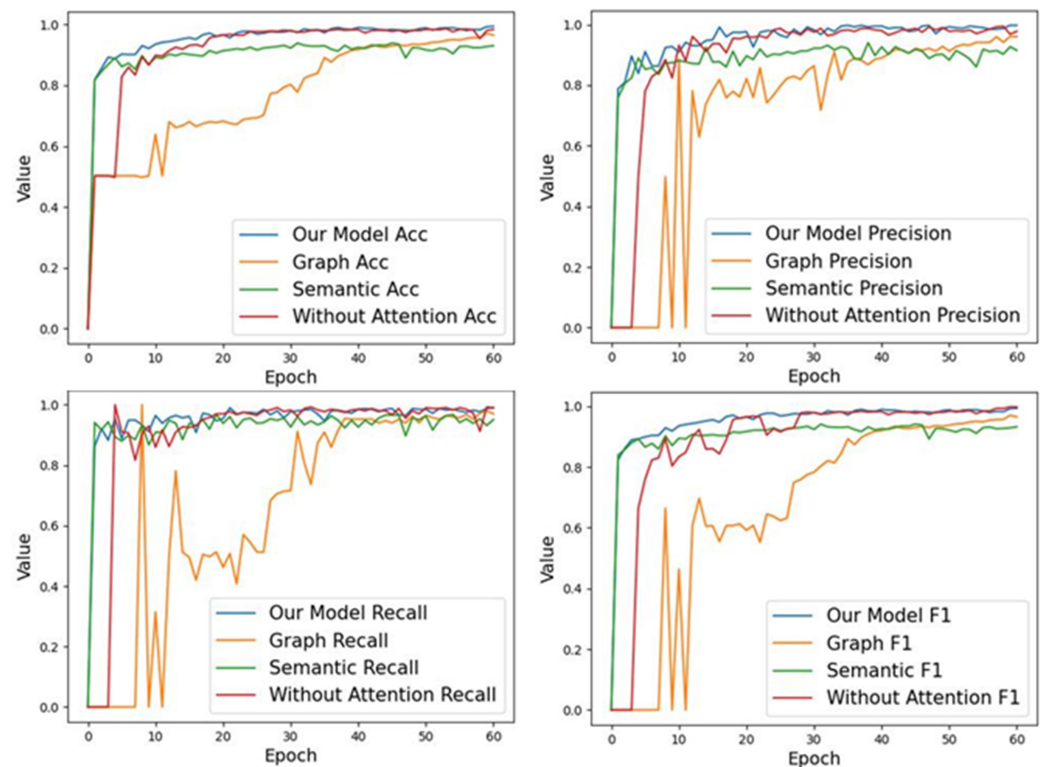


Figure 5. Acc, Precision, Recall and F1 training results.

To validate the method, we conducted classification tests on both Dataset A and Dataset B. Initially, we performed binary classification on Dataset A using a self-designed four-layer Convolutional Neural Network for image feature extraction training. The training basically converged after 60 rounds. The experiment shows that only using image features and only using semantic features can achieve 97% and 92% accuracy, respectively. This indicates robust classification performance for API sequences, whether extracting image features or semantic features. Our proposed method combines image features with semantic features and finally achieves an accuracy of 99.25%, a significant improvement compared to using a single module. Additionally, other indicators are superior to those of other models. Considering the dataset includes infrequent or nearly nonexistent API calls, resulting in pixels at these positions being set to 0, we appropriately reduce attention to this part. Therefore, in the image features, we introduce an attention mechanism to make it focus on more critical information. Applying the attention mechanism improves classification performance by approximately 1%. The method of mapping API call sequences to grayscale images proves effective, as the generated images reflect the frequency of each API in the sequence and the call relationships between every two APIs. These are crucial features influencing the classification results. We extract these features through the form of images using the superiority of Convolutional Neural Networks in images and obtain better model performance.

Additionally, using TextCNN and LSTM, two sequence-based neural networks, to process the text information of API call sequences is a suitable choice. By convolving API sequences with various convolution kernels, we extract semantic features from different perspectives that reflect the sequential characteristics among multiple API calls. Unlike directly extracting text features using N-gram, our method performs convolution on word embeddings, establishing more connections between APIs. Our approach encapsulates features such as API call frequency, call numbers, call relationships, and connections between APIs in the API call sequence. These features are processed by suitable neural networks, leading to a high accuracy rate.

Next, we explore the impact of increased threshold n for pairwise calls on training results when mapping the API sequence into the grayscale image module. Here, we set n to 1, 5, 10, and 20. After 60 rounds of training, the classification results on the test set are shown in Table 4.

Table 4. Comparison of different threshold values (n).

Value of Increased Pixels (n)	Accuracy	Recall	Precision	F-Measure
1	92.34	91.30	93.80	92.53
5	93.75	91.87	95.64	93.72
10	94.63	93.24	96.26	94.72
20	99.38	99.74	98.98	99.36

As illustrated in Figure 6, as the pixel value of the grayscale image increases, the final classification performance gradually improves. When we set the threshold n to 1 (if there is a call between A and B, the corresponding value in the matrix is increased by 1, i.e., the pixel value of the final grayscale image is increased by 1), the classification effect is the worst, only 92.38% accuracy. The accuracy improves significantly with higher values of n . Specifically, when n is 5 and 10, the accuracy can reach 93.75% and 94.62%, which is a significant improvement, and the precision, recall, and F1 scores are also rising. In our approach, we choose to set n to 20 and obtain an accuracy of 99.38. This decision is influenced by the preprocessing steps applied to the dataset, leading to a substantial reduction in repeated API calls. Setting a larger value can better represent the API call relationship and increase the discrimination. However, if n is set too large or the number of repeated calls of the data set is too large, there will be many cases where the value in the two-dimensional matrix exceeds 255. At this time, in the gray image, we regard it as 255, which will lead to the neglect of part of the same API pairwise call relationship and affect the classification results.

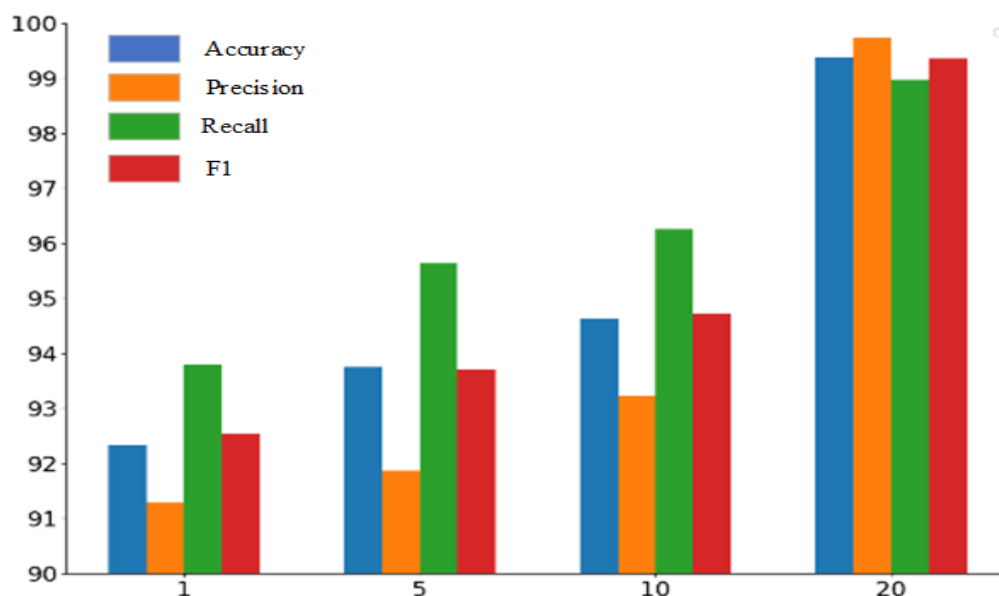


Figure 6. Comparison of different threshold values (n).

We evaluated the impact of sequence truncation on malware detection using Dataset A. During preprocessing, we removed a large number of consecutive duplicate API calls, and it was necessary to find an appropriate truncation point to reflect the behavior of the malware. Truncating too early may result in the loss of critical behavioral information,

preventing the model from fully capturing the patterns of malicious behavior. On the other hand, truncating too late may lead to the model processing more redundant or irrelevant information, introducing higher levels of noise that affect the experimental results. We tested truncation points at 200, 500, 800, 1000, and 1500 to find the best truncation point for classification performance. Table 5 shows the classification results after 40 training epochs on Dataset A. As the truncation point moved closer to 1000, the classification performance improved, reaching the highest accuracy of 99.38% at truncation point 1000. However, when the truncation point was set to 1200, the classification performance decreased. This suggests that after preprocessing the original sequences, a truncation point at 1000 already provides a sufficiently large time span to reflect the malicious behavior. When the truncation point is below 1000, many behaviors have not been fully captured. When the sequence is truncated beyond 1000, most of the malicious behavior has already ended, and benign operations such as closing handles and releasing memory, which is common to both benign and malicious activities, begin to dominate, negatively impacting the classification performance.

Table 5. Comparison of different sequence truncation.

Value of Sequence Truncation	Accuracy	Recall	Precision	F-Measure
200	94.00	94.54	93.61	94.07
500	96.25	94.84	99.05	96.42
800	97.25	95.83	99.61	97.16
1000	99.38	99.74	98.98	99.36
1200	98.75	99.05	98.60	98.80

To better validate the effectiveness of the method, we use the same method to test the effect of multiple classification tasks on Dataset B. In multiple classifications, we need more detailed features. Therefore, we tried various networks for image processing, such as VGG16 and AlexNet. Using transfer learning, we take the pre-trained network on ImageNet and use the fine-tuned network as the image feature extraction module. In the image preprocessing, we adjust and copy the image, and finally use $224 \times 224 \times 3$ as the input of the neural network.

First, we compared different feature extraction methods. In the semantic information extraction module, we use a GRU, similar to an LSTM. GRU is an improved type of Recurrent Neural Network that has fewer parameters and faster training speed. In the image information extraction module, we experiment with VGG and AlexNet. After training for 40 epochs, the experimental results are shown in Table 6.

As shown in the table, the combination of VGG and Bi-LSTM yields the best performance in the multi-classification task, achieving an accuracy of 99.58%. For both Bi-LSTM and Bi-GRU, the classification accuracy surpasses that of unidirectional models. This indicates that in API call sequences, both forward and backward directions enhance the sequence features. Therefore, both directions should be considered. In the feature extraction module, GRU, due to its fewer parameters, performs worse in classification compared to LSTM. LSTM exhibits a greater advantage in Backdoor classification. Regarding the image feature extraction module, VGG demonstrates high accuracy and effectively completes the classification task. In contrast, AlexNet is nearly incapable of performing it.

Next, we investigated the impact of transfer learning on the classification task in this experiment. Since AlexNet was unable to complete the classification task, it was not considered further. We compared the classification performance of different network combinations with and without transfer learning. The experimental results are shown in Table 7.

Table 6. Comparison of different networks on Dataset B.

		VGG+ BiLSTM	VGG+ LSTM	VGG+ BiGRU	VGG+ GRU	AlexNet+ BiGRU	AlexNet+ BiLSTM
ACC		0.9958	0.9790	0.9923	0.9867	0.6188	0.6258
Adware	Precision	1	1	1	1	0.9242	0.7901
	Recall	1	1	1	1	0.8026	0.8421
	F-Measure	1	1	1	1	0.8592	0.8153
Backdoor	Precision	1	0.9710	0.9756	0.9571	0.6581	0.6184
	Recall	1	1	0.9950	1	0.5075	0.6368
	F-Measure	1	0.9853	0.9852	0.9781	0.5730	0.6275
Downloader	Precision	1	0.9343	1	1	0.7733	0.7277
	Recall	1	0.9900	1	0.9751	0.5771	0.6915
	F-Measure	1	0.9614	1	0.9874	0.6610	0.7092
Dropper	Precision	0.9832	0.9734	0.9831	0.9615	0.6139	0.5164
	Recall	0.9832	0.9832	0.9721	0.9777	0.5419	0.6145
	F-Measure	0.9832	0.9778	0.9775	0.9695	0.5757	0.5612
Spyware	Precision	0.9824	0.9880	0.9824	1	0.5053	0.4681
	Recall	1	0.9820	1	0.9880	0.5749	0.5269
	F-Measure	0.9911	0.9850	0.9911	0.9940	0.5378	0.4958
Trojan	Precision	1	0.9851	1	1	0.3812	0.5448
	Recall	0.9851	0.9851	0.9751	0.9652	0.6468	0.3930
	F-Measure	0.9925	0.9851	0.9874	0.9823	0.4797	0.4566
Virus	Precision	1	1	1	0.9852	0.8515	0.8757
	Recall	1	0.9303	1	0.9950	0.8557	0.8060
	F-Measure	1	0.9639	1	0.9901	0.8536	0.8394
Worm	Precision	1	1	1	1	0.6606	0.5668
	Recall	1	0.9751	1	1	0.5423	0.6119
	F-Measure	1	0.9874	1	1	0.5956	0.5885

Table 7. Comparison for transfer learning on Dataset B.

	VGG+BiGRU +Transfer Learning	VGG+BiLSTM +Transfer Learning	VGG+ BiGRU	VGG+ BiLSTM
Time (s)	2966	3366	3569	4050
ACC(%)				
Epoch1	31.75	34.48	14.09	14.09
Epoch10	96.07	98.46	33.57	17.73
Epoch20	98.69	98.88	96.97	36.93
Epoch30	98.95	99.09	99.01	98.44
Epoch40	99.23	99.58	99.16	99.51

Table 7 displays the total ACC values for every 10 epochs and the time taken to train the model for 40 epochs. As shown in the table, the model using transfer learning achieved over 98% ACC earlier than the model without transfer learning. Furthermore, the use of transfer learning decreased the time required to complete 40 epochs of the classification task. With pre-trained parameters, the image feature extraction process was over 600 s faster per epoch compared to not using transfer learning. This led to a total time saving of ten minutes over 40 epochs. In terms of accuracy, the use of transfer learning had little impact on the model's classification performance, with both approaches achieving over 99%. However, the transfer learning method demonstrated faster convergence and reduced training time.

After 40 epochs of training, the testing results and confusion matrix are shown in Table 8 and Figure 7. Training data show that our approach has a very high recog-

dition rate for each of the malware families. It is outstanding in the recognition of the two categories of Adware and Downloader. However, the recognition rate of the Dropper family is relatively low, and still four failed to classify successfully. A large number of droppers were misclassified as Trojans, possibly because the two share many overlapping functions. Some droppers, when executing malicious software, even perform functions similar to those of Trojans. For example, both may disguise themselves to trick users into downloading and executing them, and their malicious behaviors often involve downloading and executing harmful code. During feature extraction, the differences between the two types were likely confused. To solve this problem, we think we can try to introduce memory features to further optimize the classification results.

Table 8. Classification results of malware families

Malware Family	Accuracy	Recall	Precision	F-Measure
Adware	1.0	1.0	1.0	1.0
Backdoor	0.99	0.99	0.99	0.99
Downloader	1.0	1.0	1.0	1.0
Dropper	0.99	0.97	1.0	0.98
Spyware	0.99	1.0	0.98	0.99
Trojan	0.99	0.99	0.99	0.99
Virus	0.99	0.99	1.0	0.99
Worms	0.99	1.0	0.99	0.99

Finally, Figures 8 and 9 present the classification performance evaluation of Datasets A and B using ROC curves and AUC values. The ROC curve, by displaying the True Positive Rate (TPR) and False Positive Rate (FPR) of the model at different thresholds, provides an intuitive visualization of the performance of both datasets. The ROC curves for Datasets A and B shown in the figure demonstrate good classification ability, indicating the model's strong robustness on both datasets.

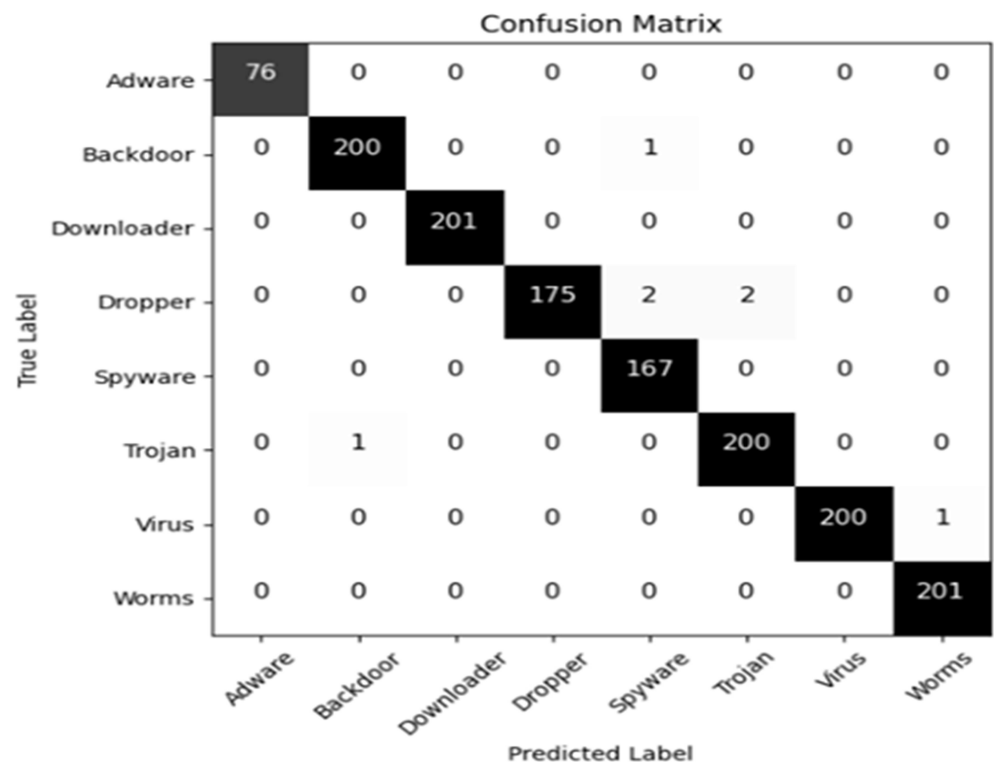


Figure 7. Confusion Matrix of malware families.

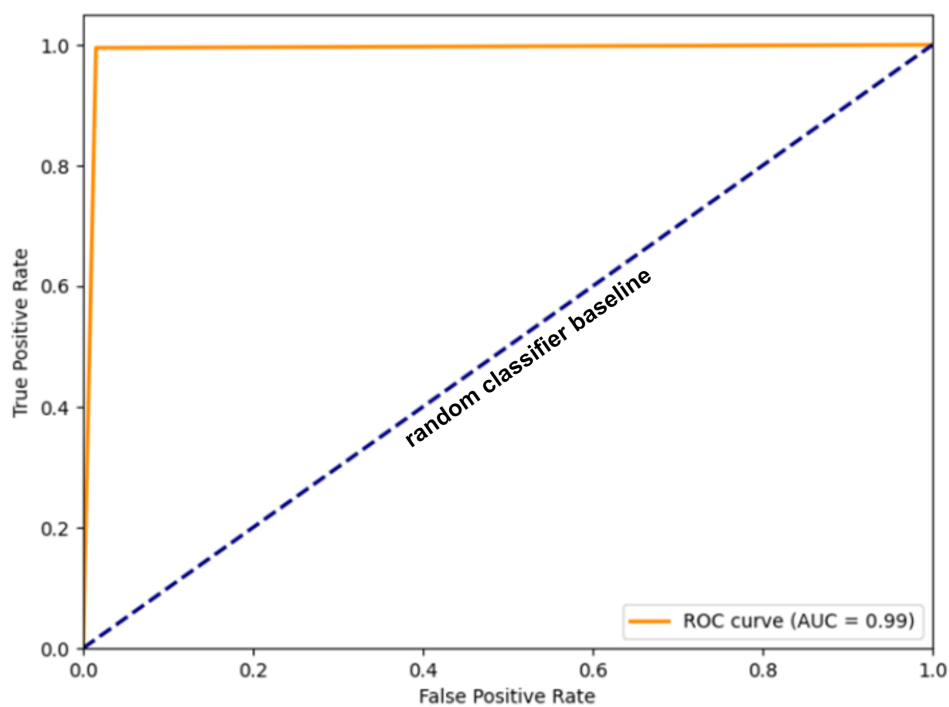


Figure 8. ROC Curve For Dataset A.

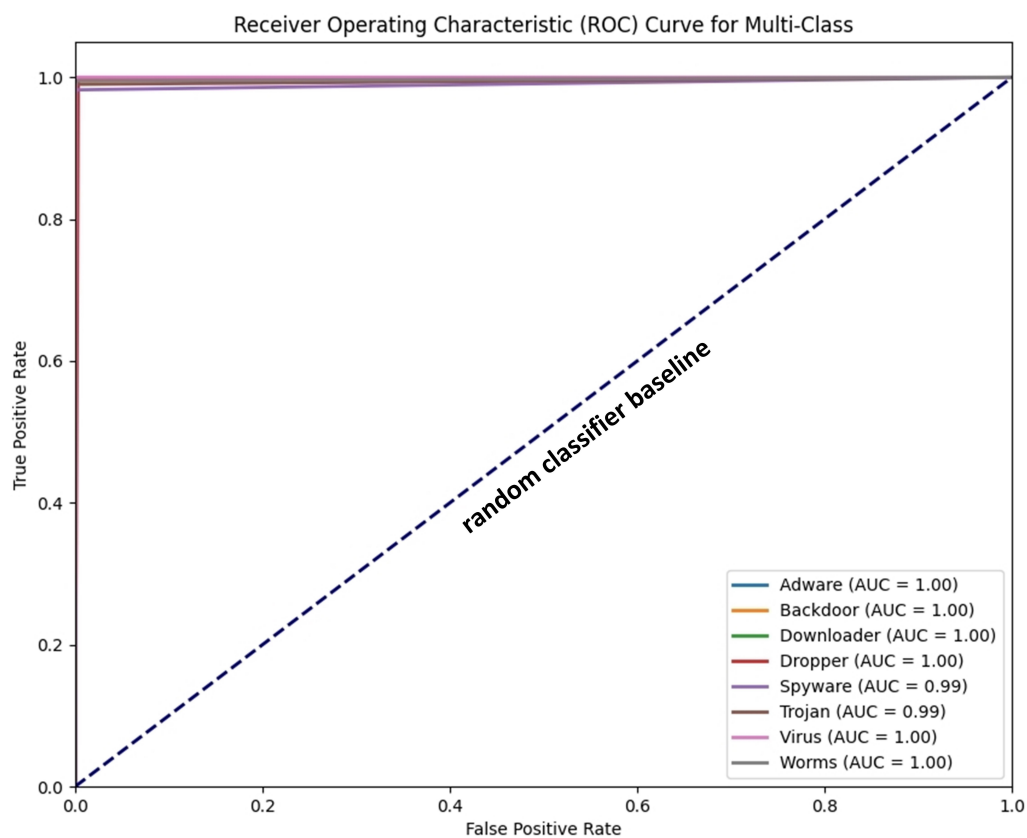


Figure 9. Roc Curve For Dataset B.

In our final comparison with other studies focused on API call sequences, we replicated the methods outlined in [5,28] on Dataset A. After dividing the dataset into training and

test sets, we presented the final classification results. Table 9 illustrates the outstanding performance of our method in terms of metrics, achieving the highest indicators.

Table 9. Comparison of different studies.

Study	Accuracy	Recall	Precision	F-Measure
Tran et al. [28]	0.9463	0.9339	0.9515	0.9426
Li et al. [5]	0.9487	0.9529	0.9458	0.9493
Our model	0.99	0.9853	0.9951	0.9902

7. Conclusions and Future Work

Malware occurrences continue to be one of the issues in network security that cannot be disregarded because of their high frequency and severe effects. In this research, we present a malware-detection method that uses API call sequences and deep learning to perform binary malware categorization. To achieve successful detection, the method entails building a neural network model and choosing an appropriate deep-learning network. Using the advantages of Convolutional Neural Networks in image classification, we extract features of grayscale images from sequences of API calls. Furthermore, we leverage the sequence task-handling capabilities of TextCNN and LSTM to extract semantic features and sequence relationships from API call sequences from different angles. Lastly, we use the fully linked layer categorization to find the harmful code included in the original software. Our approach fully utilizes the properties of various neural networks for analyzing text or image sequences. We enhance the feature space for malware classification by extracting a wide range of features from API sequences. These characteristics include the number of API calls, the linkages between APIs, the frequency of API occurrences, and the interactions within API call sequences.

A major obstacle to our strategy is the APT attacks. Long latency times, lengthy API call sequences, and high persistence are common traits of APT attacks. Even when a number of preprocessing steps are taken, it is still challenging to capture the APT attack load at the end of the API sequence, and it will take a significant amount of training time. After eliminating unnecessary API calls, we will attempt to segment the API call sequence in further work to address this issue. Furthermore, we want to iteratively identify particular areas in order to optimize the procedure. This method ensures thorough coverage of API sequences throughout the lengthy sequence by concentrating on a specific segment during each detection iteration. Additionally, APT attacks often use sophisticated evasion techniques, such as anti-virtual machine (VM) measures, to avoid detection. When running in a virtualized environment, certain malicious behaviors may remain hidden or altered, making them difficult to detect through dynamic analysis. In order to mitigate the effects of anti-virtual machine technology on dynamic analysis, we will use static analysis in our upcoming work. This will be integrated with memory forensics techniques to record temporary content creation and aid in the detection of off-ground attacks. We will also investigate how to create more realistic sandbox environments. Additionally, we plan to extend our work to real-time detection, ensuring that identifiable information is replaced with anonymous identifiers, therefore preventing the exposure of any personal or sensitive data during the detection process.

Another direction for our future work is to explore security models based on the content of this paper. We will standardize and modularize the entire process and study adversarial training by continuously simulating potential attacks to enhance adversarial robustness. At the same time, we will actively deploy our solution in real-world scenarios, continuously improving the overall process and security performance and striving for the effectiveness and timeliness of real-time detection.

Author Contributions: Conceptualization, S.Z. and M.G.; methodology, S.Z.; software, M.G.; validation, S.X.; formal analysis, M.G.; investigation, M.G.; data curation, M.G.; writing—original draft preparation, S.Z. and M.G.; writing—review and editing, W.S. and R.K.; visualization, M.G.; supervision, L.W.; project administration, L.W.; funding acquisition, S.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China (62102209); the Taishan Scholars Program (tsqn202312231); the Shandong Provincial Natural Science Foundation of China (ZR2024MF104); the Shandong Provincial Key Research and Development Program (2021CXGC010107); the New 20 project of higher education of Jinan, China (202228017).

Data Availability Statement: The datasets used during the current study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. AV-TEST. Malware Statistics & Trends Report, AV Test Malware Statistics. Available online: <https://www.av-test.org/en/statistics/malware> (accessed on 18 December 2023).
2. AV-ATLAS. Malware & PUA. Available online: <https://portal.av-atlas.org/malware> (accessed on 18 December 2023).
3. Huang, X.; Ma, L.; Yang, W.; Zhong, Y. A method for windows malware detection based on deep learning. *J. Signal Process. Syst.* **2021**, *93*, 265–273. [CrossRef]
4. Shenderovitz, G.; Nissim, N. Bon-APT: Detection, attribution, and explainability of APT malware using temporal segmentation of API calls. *Comput. Secur.* **2024**, *142*, 103862. [CrossRef]
5. Li, C.; Lv, Q.; Li, N.; Wang, Y.; Sun, D.; Qiao, Y. A novel deep framework for dynamic malware detection based on API sequence intrinsic features. *Comput. Secur.* **2022**, *116*, 102686. [CrossRef]
6. Vasan, D.; Alazab, M.; Wassen, S.; Naeem, H.; Safaei, B.; Zheng, Q. IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture. *Comput. Netw.* **2020**, *171*, 107138. [CrossRef]
7. Darabian, H.; Dehghantanha, A.; Hashemi, S.; Homayoun, S.; Choo, K.K.R. An opcode-based technique for polymorphic Internet of Things malware detection. *Concurr. Comput. Pract. Exp.* **2020**, *32*, e5173. [CrossRef]
8. Qiang, W.; Yang, L.; Jin, H. Efficient and robust malware detection based on control flow traces using deep neural networks. *Comput. Secur.* **2022**, *122*, 102871. [CrossRef]
9. Zhang, S.; Hu, C.; Wang, L.; Mihaljevic, M.J.; Xu, S.; Lan, T. A Malware Detection Approach Based on Deep Learning and Memory Forensics. *Symmetry* **2023**, *15*, 758. [CrossRef]
10. Zhou, L.; Zhang, F.; Xiao, J.; Leach, K.; Weimer, W.; Ding, X.; Wang, G. A coprocessor-based introspection framework via intel management engine. *IEEE Trans. Dependable Secur. Comput.* **2021**, *18*, 1920–1932. [CrossRef]
11. Yang, F.; Xu, J.; Xiong, C.; Li, Z.; Zhang, K. {PROGRAPHER}: An Anomaly Detection System based on Provenance Graph Embedding. In Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23), Anaheim, CA, USA, 9–11 August 2023; pp. 4355–4372.
12. Wang, Q.; Hassan, W.U.; Li, D.; Jee, K.; Yu, X.; Zou, K.; Rhee, J.; Chen, Z.; Cheng, W.; Gunter, C.A.; et al. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In Proceedings of the NDSS, San Diego, CA, USA, 23–26 February 2020.
13. Tan, X.; Zhao, Z. SHERLOC: Secure and Holistic Control-Flow Violation Detection on Embedded Systems. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Copenhagen, Denmark, 26–30 November 2023; pp. 1332–1346.
14. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware images: Visualization and automatic classification. In Proceedings of the Proceedings of the 8th International Symposium on Visualization for Cyber Security, Pittsburgh, PA, USA, 20 July 2011 ; pp. 1–7.
15. Bensaoud, A.; Kalita, J. CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls. *Knowl.-Based Syst.* **2024**, *290*, 111543. [CrossRef]
16. Shah, S.S.H.; Jamil, N.; ur Rehman Khan, A.; Sidek, L.M.; Alturki, N.; Zain, Z.M. MalRed: An innovative approach for detecting malware using the red channel analysis of color images. *Egypt. Inform. J.* **2024**, *26*, 100478. [CrossRef]
17. Shaikat, K.; Luo, S.; Varadharajan, V. A novel deep learning-based approach for malware detection. *Eng. Appl. Artif. Intell.* **2023**, *122*, 106030. [CrossRef]
18. Obaidat, I.; Sridhar, M.; Pham, K.M.; Phung, P.H. Jadeite: A novel image-behavior-based approach for Java malware detection using deep learning. *Comput. Secur.* **2022**, *113*, 102547. [CrossRef]
19. Jha, S.; Prashar, D.; Long, H.V.; Taniar, D. Recurrent neural network for detecting malware. *Comput. Secur.* **2020**, *99*, 102037. [CrossRef]

20. Gómez, A.; Muñoz, A. Deep Learning-Based Attack Detection and Classification in Android Devices. *Electronics* **2023**, *12*, 3253. [[CrossRef](#)]
21. Amer, E.; Zelinka, I. A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. *Comput. Secur.* **2020**, *92*, 101760. [[CrossRef](#)]
22. Li, N.; Lu, Z.; Ma, Y.; Chen, Y.; Dong, J. A Malicious Program Behavior Detection Model Based on API Call Sequences. *Electronics* **2024**, *13*, 1092. [[CrossRef](#)]
23. Nawaz, M.S.; Fournier-Viger, P.; Nawaz, M.Z.; Chen, G.; Wu, Y. MalSPM: Metamorphic malware behavior analysis and classification using sequential pattern mining. *Comput. Secur.* **2022**, *118*, 102741. [[CrossRef](#)]
24. Qian, L.; Cong, L. Channel Features and API Frequency-Based Transformer Model for Malware Identification. *Sensors* **2024**, *24*, 580. [[CrossRef](#)] [[PubMed](#)]
25. Lai, Y.; Zhang, L. Government affairs message text classification based on RoBERTa and TextCNN. In Proceedings of the 2023 5th International Conference on Communications, Information System and Computer Engineering (CISCE), Guangzhou, China, 14–16 April 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 258–262.
26. Chen, X.; Hao, Z.; Li, L.; Cui, L.; Zhu, Y.; Ding, Z.; Liu, Y. Cruparamer: Learning on parameter-augmented api sequences for malware detection. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 788–803. [[CrossRef](#)]
27. Catak, F.O.; Yazı, A.F. A benchmark API call dataset for windows PE malware classification. *arXiv* **2019**, arXiv:1905.01999.
28. Tran, T.K.; Sato, H. NLP-based approaches for malware classification from API sequences. In Proceedings of the 2017 21st Asia Pacific Symposium on Intelligent and Evolutionary Systems (IES), Hanoi, Vietnam, 15–17 November 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 101–105.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.